

Sieci Komputerowe



⌘ Wykład

Bezpieczeństwo w sieciach
komputerowych.

Wybrane zagadnienia.

Materiały do Laboratoriów:
NET ACCESS

Bezpieczeństwo w sieciach – obszary zainteresowania



- ⌘ Problematykę bezpieczeństwa w sieciach komputerowych można koncentrować wokół zagadnień:
 - ☒ Utajniania informacji (*secrecy*) – z utrzymaniem jej poza zasięgiem osób nieuprawnionych
 - ☒ Autentyfikacji uczestników komunikacji (*authentication*) – z wykazaniem autentyczności rozmówców
 - ☒ Dowodzenia faktu dokonania czynności (*nonrepudiation*) – z możliwością udowodnienia uczestnikowi komunikacji, iż faktycznie dokonał określonych czynności
 - ☒ Dowodzenia autentyczności otrzymanej informacji (*integrity control*) – z wykazaniem, iż informacja pochodząca oryginalnie z określonego systemu nie została zmodyfikowana lub sfabrykowana w całości

Kryptografia, definicje



- ⌘ Kryptografia - gałąź wiedzy i badań zajmująca się utajnieniem zapisywaniem
- ⌘ Szyfrowanie, kodowanie - proces przekształcania tekstu jawnego w tekst zaszyfrowany (kryptogram),
- ⌘ Odszyfrowanie, dekodowanie - proces odwrotny
- ⌘ Tekst jawny (*plaintext*) i kryptogram (*ciphertext*) są podzielone na jednostki tekstu (pojedyncza litera, para liter (digram), trójka liter (trigram), blok składający się N liter)
- ⌘ W praktyce stosowane są dwa podstawowe rodzaje szyfrów: przestawieniowe i podstawieniowe
- ⌘ Artefakty: algorytm szyfrowania E i algorytm deszyfrowania D - nie ingerujące w specyfikę językową szyfrowanej informacji (*cipher*), tajny klucz (*key*) lub klucze. Gdy algorytm E i D interpretuje słowa języka (np. podmienia całe wyrazy język naturalnego) – nazywamy go kodem (*code*)

Kryptografia, definicje



- ⌘ Kryptoanaliza - dziedzina wiedzy i badań zajmująca się metodami przełamывania szyfrów.
- ⌘ Szyfr przełamывalny - szyfr przy którym jest możliwość odtworzenia tekstu jawnego bądź klucza na podstawie tekstu zaszyfrowanego (kryptogramu), ewentualnie określenia klucza na podstawie znajomości tekstów jawnego i zaszyfrowanego.
- ⌘ Kryptologia - obejmuje zarówno kryptografię jak i kryptoanalizę.
- ⌘ Kryptosystem – zbiór procedur umożliwiający świadczenie usług z zastosowaniem technik kryptograficznych, polegających na utajnianiu przesyłanej lub przechowywanej informacji

Poufność, autentyczność, rodzaje kryptosystemów

- ⌘ Podstawowe warunki, jakie musi spełnić kryptosystem:
 - ☒ Poufność - nie jest obliczeniowo możliwe ustalenie choćby fragmentu zaszyfrowanej wiadomości na podstawie szyfrogramu
 - ☒ Autentyczność - nie jest możliwa niewykrywalna zmiana zaszyfrowanej informacji na jej drodze do uprawnionego odbiorcy
- ⌘ Kryptosystemy dzielimy na:
 - ☒ symetryczne - jednakowe klucze szyfrowania i deszyfrowania (z wykorzystaniem *symmetric-key algorithms*)
 - ☒ asymetryczne - para kluczy (najczęściej prywatny i publiczny)
 - ☒ hybrydowe - mieszanie w/w technik
- ⌘ Reguła Kerckhoffa dotycząca kryptosystemów:
„Wszystkie algorytmy muszą być jawne, tylko klucze są utajnione”

Szyfrowanie hybrydowe - przykład sesji



- ⌘ Kryptosystem asymetryczny - używany na potrzeby wymiany kluczy (tzw. klucza sesyjnego)
- ⌘ Kryptosystem symetryczny – służy do szyfrowania właściwych danych wspólnym (wymienionym przy użyciu poprzedniej techniki) kluczem
- ⌘ Sesja wymiany informacji:
 - ☒ Strona inicjująca komunikację generuje losowo klucz sesyjny,
 - ☒ Strona inicjująca szyfruje treść klucza za pomocą publicznego klucza odbiorcy
 - ☒ Odbiorca odszyfrowuje klucz sesyjny swoim kluczem prywatnym i informuje nadawcę o gotowości do przesyłu danych. Analogicznie w przeciwnym kierunku. Dalsza interakcja stron odbywa się na zasadzie algorytmu symetrycznego.

Dwie zasady projektowania bezpiecznych kryptosystemów

⌘ Redundancja (*redundancy*):

Informacja zaszyfrowana losowym kluczem i podstawiona do systemu przez atakującego nie może być przypadkowo zinterpretowana jako poprawna (rutynowe dodanie jawnej i redundantnej informacji do tekstu jawnego przed zaszyfrowaniem uniemożliwi przypadkowe otrzymanie komunikatu poprawnego po odszyfrowaniu)

⌘ Aktualność czasowa informacji (*freshness*):

Informacja zaszyfrowana, przechwycona przez atakującego i wysłana do systemu w późniejszym terminie (bez zmian) nie może być zinterpretowana jako poprawna (rutynowe dodanie sygnatury czasowej, numeru sekwencji lub podobnych danych do tekstu jawnego przed zaszyfrowaniem uniemożliwi przypadkowe otrzymanie komunikatu poprawnego po rozszyfrowaniu)

Przykład techniki utajniania informacji

- ⌘ *One-Time pads* – technika polega na wykonaniu następujących kroków:
 - ☒ Zakodowanie informacji do utajnienia w postaci ciągu bitów o znanej długości (np. reprezentacji ASCII w przypadku tekstu)
 - ☒ Wylosowanie ciągu bitów o identycznej długości
 - ☒ Wykonanie operacji XOR pomiędzy dwoma powyższymi ciągami bitów (powstaje szyfrogram)
- ⌘ Odszyfrowanie informacji wymaga tego samego klucza (należy go przekazać stronie odbierającej szyfrogram), która ponownie wykona operację XOR na szyfrogramie (odzyskując informację wcześniej zaszyfrowaną)
- ⌘ Możliwe jest wielokrotne użycie wylosowanego ciągu bitów dla kolejnych danych utajnianych (osłabia to jednak siłę szyfru)
- ⌘ Technika *One-Time pads* jest elementem wielu dzisiejszych kryptosystemów

Popularne szyfry - Szyfr DES



- ⌘ DES - Data Encryption Standard
- ⌘ Bazuje na algorytmie DES (podstawiania i permutacji)
- ⌘ Szyfrowane są 64 bitowe porcje informacji. Klucz ma 56 bitów. Klucz jest symetryczny
- ⌘ Algorytm:
 - ⊞ Zamiana kolejności bitów w sekwencji szyfrowanej - na podstawie klucza
 - ⊞ Szesnastokrotne wykonanie kodowanej kluczem funkcji szyfrującej, która dokonuje podstawień bitów i następnie zamienia miejscami połowę starszą i młodszą bloku danych szyfrowanych
- ⌘ 3DES (*Triple DES*), DESX - wersje ulepszone DES - zwiększona długość klucza, wielokrotne szyfrowanie wariantami klucza itp.
- ⌘ Wariant wzmocnienia DES o nazwie *whitening*: wykonanie *One-Time pads* przed szyfrowaniem i następnie ponownie XOR przed wysłaniem

Popularne szyfry - Szyfr 3DES



- ⌘ Wzmocniony wariant szyfru DES
- ⌘ Tworzone są trzy klucze (K_1 , K_2 , K_3), po kolei wykorzystywane
- ⌘ Proces szyfrowania składa się z szyfrowania kluczem K_1 , następnie użycia procedury odszyfrowania kluczem K_2 , i ponownie szyfrowania kluczem K_3
- ⌘ Proces odszyfrowania składa się z odszyfrowania kluczem K_1 , następnie użycia procedury szyfrowania kluczem K_2 , i ponownie odszyfrowania kluczem K_3
- ⌘ 3DES jest popularny w implementacjach produkcyjnych

Popularne szyfry - Szyfr IDEA



- ⌘ IDEA - International Data Encryption Algorithm
- ⌘ Pierwsze opracowanie - rok 1990,
- ⌘ Szyfrowanie 64-bitowych bloków tekstu jawnego,
- ⌘ Klucz 128 bitowy, symetryczny
- ⌘ Polega na naprzemiennym mnożeniu modulo 2 przez bajty klucza oraz dodawaniu i mnożeniu modulo różnych bajtów z szyfrowanego bloku,
- ⌘ Kolejność tych operacji gwarantuje brak utraty informacji i możliwość przeprowadzenia procedury odwrotnej, dającej na podstawie kryptogramu tekst pierwotny.

Popularne szyfry - Szyfr AES



- ⌘ AES - Advanced Encryption Standard
- ⌘ Popularność zyskał w 1997 r, kiedy wygrał otwarty konkurs
- ⌘ Klucz 128,192 lub 256 bitowy, Symetryczny
- ⌘ Operuje on na blokach danych o długości 128 bitów
- ⌘ Procedura szyfrowania wykonuje 10 rund szyfrujących, składających się z substytucji wstępnej (podmiana wg klucza), permutacji macierzowej (mieszanie wierszy, mieszanie kolumn bloku danych) oraz modyfikacji danych wybraną kluczem funkcją

Popularne szyfry - Szyfr Blowfish



- ⌘ Szyfrowanie 64-bitowych bloków tekstu jawnego,
- ⌘ Wielkość klucza - od 32 do 448 bitów,
- ⌘ Technika S-BOX (Substitution Box), której namiastka wykorzystywanabyła też w DES. Polega na podstawieniu ciągu bitów innym, w zależności od bitów: poprzedzającego i następującego
- ⌘ Algorytm wykonuje 16 rund podstawiania S-BOX

Problem stałej długości bloku szyfrowanego

- ⌘ Problem: Można z przechwyconych zaszyfrowanych bloków (o stałej długości i nieznanej treści) skomponować nową poprawną zaszyfrowaną informację.
- ⌘ Rozszerzenia algorytmów szyfrowania zakładają nast. rozwiązania:
 - ☒ *Cipher Block Chaining* – każdy blok tekstu jawnego przed zaszyfrowaniem/odszyfrowaniem jest modyfikowany operacją XOR z poprzednim zaszyfrowanym już blokiem (powstaje powiązanie poprzedniego i bieżącego bloku uniemożliwiające niezauważoną zamianę bloków w szyfrogramie)
 - ☒ *Cipher Feedback Mode* – procedura prowadzona analogicznie do *Cipher Block Chaining* z tą różnicą, że jako drugi operand XOR stosowany jest 64-bitowy bufor przesuwany zawierający ostatnie 8 bajtów przetwarzanego szyfrogramu (przy odszyfrowaniu można ją wykonywać z już każdym otrzymanym bajtem szyfrogramu, a nie całym blokiem)

Problem stałej długości bloku szyfrowanego

- ⌘ Dwie poprzednie techniki wymagają odszyfrowania całej informacji znajdującej się w strumieniu danych przed tą, będącą przedmiotem zainteresowania (np. fragment zaszyfrowanego pliku). Dlatego opracowano kolejne rozszerzenie o nazwie *Counter Mode*. Rozszerzenie to polega na szyfrowaniu kluczem zmiennego z każdym blokiem i obliczalnego wektora. Następnie – na wykonaniu operacji XOR pomiędzy zaszyfrowanym wektorem a odpowiadającym mu fragmentem tekstu jawnego. Wynik jest szyfrogramem. Odszyfrowanie dowolnego fragmentu szyfrogramu wymaga obliczenia odpowiadającej mu wartości wektora, zaszyfrowania go i ponownego wykonania operacji XOR z szyfrogramem. Nie jest wtedy wymagana odszyfrowanie wszystkich danych poprzedzających dany blok, a nieuprawniona zamiana bloków w szyfrogramie zostanie wykryta

Algorytm RSA

– public key cryptography

- ⌘ Algorytm z kluczem asymetrycznym (nazwa od nazwisk autorów: Rivest, Shamir, Adleman). Idea działania:
 - ☒ Generowanie pary kluczy (klucza publicznego KE, klucza prywatnego KD),
 - ☒ Szyfrowanie wiadomości M przy pomocy klucza publicznego KE,
 - ☒ Deszyfrowanie kryptogramu C przy pomocy klucza prywatnego KD.
- ⌘ Klucze to pary liczb (e, n) oraz (d, n) , takie że e i d są powiązane, umożliwiając:
 - ☒ Zaszyfrowanie: **$\text{szyfr} = \text{dane}^e \bmod n$**
 - ☒ Odtworzenie: **$\text{dane} = \text{szyfr}^d \bmod n$**
 - ☒ W tej sytuacji może nastąpić upublicznienie e i n przy jednoczesnym braku możliwości odtworzenia d na ich podstawie

Certyfikowanie kryptograficzne dokumentów

- ⌘ Podpisy lub sygnatury elektroniczne (*digital signatures*) dokumentów mają w swoim założeniu zastąpić przekazywanie wiadomości podpisywanych fizycznie (papier) wiadomościami przesyłanymi elektronicznie. Konieczne jest tu przesyłanie tych wiadomości tak, aby spełnione były następujące kryteria:
 - ☒ Odbiorca może zweryfikować tożsamość, którą posłużył się nadawca (i o której poinformował odbiorcę w wiadomości)
 - ☒ Nadawca nie może zaprzeczyć, że to on wysłał dany dokument
 - ☒ Odbiorca nie może zmodyfikować otrzymanego dokumentu z jednoczesnym zachowaniem możliwości udowodnienia nadawcy, że to nadawca wysłał (zmodyfikowany już) dokument

Sygnatury z użyciem klucza symetrycznego

- ⌘ System wymaga użycia jednostki centralnej (Trusted Authority), która pośredniczy w wymianie podpisanych dokumentów.
- ⌘ Dokument podpisywany jest wysyłany do Trusted Authority w postaci zaszyfrowanej kluczem nadawcy (znanym też w Trusted Authority). Cała wiadomość jest opatrzona identyfikatorem odbiorcy dokumentu i sygnaturą czasową (te dane także są zaszyfrowane). Tam następuje odszyfrowanie i zweryfikowanie poprawności formatu. Gdy wiadomość jest poprawna (udało się odszyfrować) - Trusted Authority wysyła do odbiorcy postać zaszyfrowaną kluczem znanym odbiorcy.
- ⌘ Odbiorca uzyskuje postać jawną, gdy chce udowodnić nadawcy wysłanie dokumentu – zwraca się do Trusted Authority z prośbą potwierdzenie pochodzenia wiadomości. Trusted Authority wykazuje autentyczność informacji otrzymanej od odbiorcy, uniemożliwiając też nadawcy zaprzeczenie wysłania dokumentu.

Sygnatury z użyciem klucza asymetrycznego

- ⌘ System bazujący na użyciu kryptosystemu asymetrycznego nie wymaga stosowania jednostki centralnej.
- ⌘ Dokument podpisywany jest przy użyciu klucza prywatnego nadawcy (poprzez zaszyfrowanie tym kluczem). Następnie szyfrowany jest za pomocą klucza publicznego odbiorcy – i taki szyfrogram jest do odbiorcy wysyłany. Odbiorca odszyfrowuje wiadomość przy użyciu swojego klucza prywatnego, a następnie przy pomocy klucza publicznego nadawcy (klucze publiczny i prywatny są zamienne, obydwa mogą służyć zarówno do szyfrowania jak i odszyfrowania wiadomości)
- ⌘ Odbiorca, aby udowodnić nadawcy wysłanie dokumentu, musi z powodzeniem odszyfrować wiadomość i wykazać zgodność odszyfrowanego dokumentu z ewentualnym innym, którego wysłaniu nadawca zaprzecza.

Skróty dokumentów (hash)

- ⌘ Proces tworzenia skrótu dokumentu zakłada wygenerowanie na podstawie dowolnie długiego ciągu danych łańcucha bitów o stałej długości. Funkcja realizująca ten proces (określana często mianem *message digest* lub funkcji skrótu), posiada następujące cechy:
 - ⊞ Proces obliczania skrótu dokumentu nie jest skomplikowany
 - ⊞ Nie jest możliwe otrzymanie dokumentu na podstawie jego skrótu
 - ⊞ Nie jest możliwe odzyskanie dwóch dokumentów posiadających taki sam skrót (choć teoretycznie dokumenty takie mogą istnieć, co określamy mianem kolizji funkcji skrótu)
 - ⊞ Dokonanie najmniejszej, choćby jednobitowej, zmiany dokumentu spowoduje otrzymanie innego skrótu tego dokumentu
- ⌘ W praktyce stosuje się skróty przynajmniej 128-bitowe

Sygnatury z użyciem klucza asymetrycznego i skrótu

- ⌘ System wymaga użycia kryptosystemu asymetrycznego (nie wymaga jednostki centralnej).
- ⌘ Dokument podpisywany jest przy użyciu klucza prywatnego nadawcy. W praktyce formatowany jest w ciąg bitów, z którego przy pomocy jednokierunkowej matematycznej funkcji skrótu generowana jest unikatowa wartość hash (ekstrakt wiadomości). Następnie otrzymany ekstrakt szyfrowany jest prywatnym kluczem nadawcy i tak otrzymany kryptogram (stanowiący podpis cyfrowy) dołączany jest do wysyłanej wiadomości)
- ⌘ Odbiorca, aby udowodnić nadawcy wysłanie dokumentu, musi jedynie posłużyć się jego kluczem publicznym (aby odszyfrować ekstrakt wiadomości), następnie policzyć ponownie wartość ekstraktu wiadomości otrzymanego od nadawcy w postaci jawnej. Gdy wartości są identyczne – udowodniono nadawcy wysłanie dokumentu.

Implementacje funkcji skrótu



- ⌘ MD5 – *Message Digest algorithm 5* - popularna w zastosowaniach przy identyfikacji plików i haseł (wtedy wartość wyniku ma postać 32-znakowej liczby w zapisie heksadecymalnym). Opracowano technikę generowania kolizji funkcji skrótu MD5, więc nie poleca się jej do tworzenia kryptosystemów
- ⌘ SHA-1 - *Secure Hash Algorithm 1* - tworzy 160-bitowy skrót dokumentu o maksymalnym rozmiarze 2^{64} bitów. Generuje kolizje raz na 2^{51} przypadków.
- ⌘ SHA-2 - *Secure Hash Algorithm 2* – występuje w wariantach 256/224 (tworząc skróty 256/224-bitowe z dokumentu o maksymalnym rozmiarze 2^{64} bitów) oraz 512/384 (tworząc skróty 512/384-bitowe z dokumentu o maksymalnym rozmiarze 2^{128} bitów). Jak dotąd – nie generuje kolizji.

Podpis elektroniczny dokumentu w Polsce



- ⌘ Artykuł 3, pkt 1 ustawy o podpisie elektronicznym z dnia 18 września 2001 roku:

“Podpis elektroniczny to dane w postaci elektronicznej, które wraz z innymi danymi, do których zostały dołączone, lub z którymi są logicznie powiązane, służą do identyfikacji osoby składającej podpis elektroniczny”.

- ⌘ Wykładnia:

Sekwencja znaków, która dołączona do przesyłanych danych ma stanowić podstawę do identyfikacji jego posiadacza. Jest odpowiednikiem podpisu odręcznego, powinien zatem pełnić funkcję identyfikacyjną, dowodową i kontraktową - wykorzystanie podpisu przy transakcjach pomiędzy stronami jest potwierdzeniem tożsamości podpisującego, a także jednoznacznym zaaprobowaniem umowy lub dokumentu przesłanego przez sieć komputerową.

Potwierdzanie tożsamości w sieci



- ⌘ Identyfikacja (*Identification*) czynność pozwalająca na rozróżnienie użytkowników lub urządzeń klienckich
- ⌘ Uwierzytelnienie (*Authentication*) proces weryfikacji tożsamości użytkownika. Techniki:
 - ☒ *Proof by Knowledge* : podanie hasła/kodu/sprawdzenie faktu posiadania klucza -> sprawdzenie posiadanej umiejętności (np. do odszyfrowania łańcucha znaków)
 - ☒ *Proof by Possession* : sprawdzenie poprawności posiadanego przedmiotu (np. urządzenia): karty magnetycznej w czytniku, modułu w porcie, tokenu sprzętowego etc.)
- ⌘ Autoryzacja (*Authorization*) - proces przydzielania praw (dostępu do zasobów)

Implementacje środków bezpieczeństwa w sieciach komputerowych

⌘ Wybrane technologie, protokoły i usługi

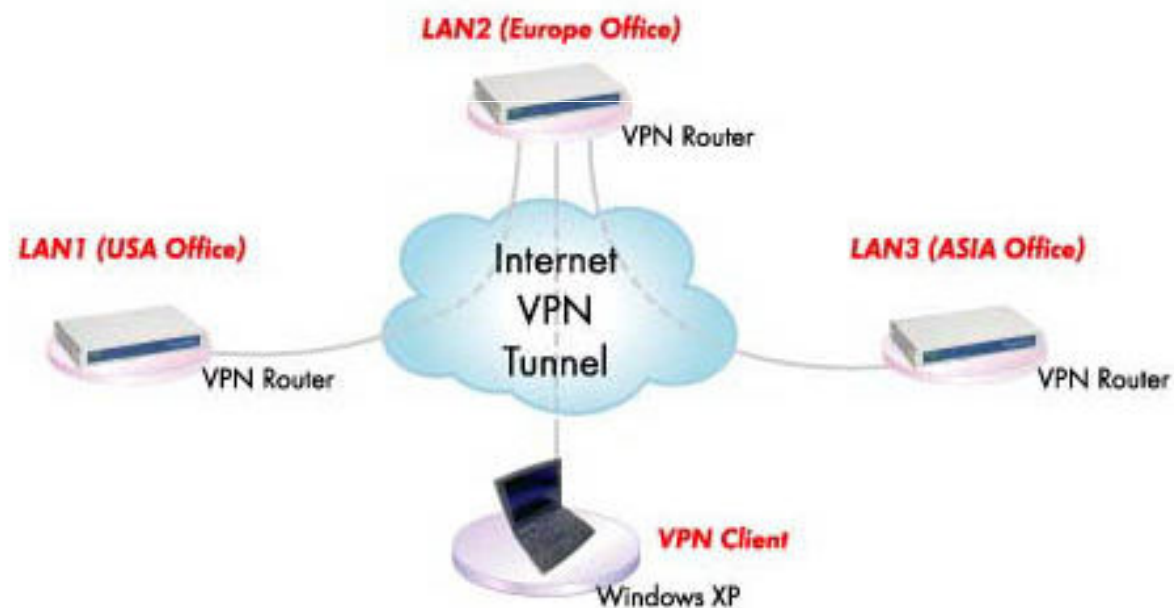
Wirtualne Sieci Prywatne (I)



- ⌘ VPN - *Virtual Private Network*. Tunele VPN istnieją w strefie publicznej (niebezpiecznej) - pomiędzy tzw. bramkami VPN prowadzącymi do podsieci prywatnych lub hostów
- ⌘ Protokoły transmisji VPN:
 - ☒ PPTP (*Point-to-Point Tunneling Protocol*) - tworzy łączy punkt-punkt zawierające kanał szyfrujący. Przeważnie stosowany do tunelowania ruchu pomiędzy całymi segmentami sieci prywatnych (o różnej adresacji). Pakiety do przeciwległej sieci są automatycznie szyfrowane i przekazywane przez tunel.
 - ☒ L2TP (*Layer2 Tunneling Protocol*) – tworzy tunel w warstwie drugiej ISO OSI, emulując funkcjonalność pseudo-wire. W praktyce stosowany częściej do podłączania pojedynczych hostów do zdalnej sieci prywatnej. Silniejszy algorytm szyfrowania (3DES, 168 bit)
 - ☒ SSTP (*Secure Socket Tunneling Protocol*) – opcja najmocniej zabezpieczona, stosuje jako klucze tunelu 2048 bitowe certyfikaty

Wirtualne Sieci Prywatne (II)

⌘ Przykład konfiguracji VPN - tunele i połączenia VPN



VPN – przykład konfigurowania (Cisco IOS)

- ⌘ Faza 1: definiowanie polityki ISAKMP (*Internet Security Association and Key Management Protocol*) dla tunelu VPN:
- ⌘ Zdefiniowanie polityki ISAMKP: *Ruter(config)#crypto isakmp policy 1* a w niej określenie:
 - ⏏ Trybu autoryzacji: *Ruter(config-isakmp)#authentication pre-share*
 - ⏏ Rodzaju klucza *hash (Secure Hash Algorithm)*:
Ruter(config-isakmp)#hash sha
 - ⏏ Algorytmu szyfrowania VPN i wielkości klucza (DES, AES, 3DES etc):
Ruter(config-isakmp)#encryption aes 128
 - ⏏ Wariantu protokołu wymiany kluczy (Diffiego-Hellmana'a) (w Cisco określane jako *grupy*):
Ruter(config-isakmp)#group 2
 - ⏏ Długości życia klucza (w sekundach, 60-86400):
Ruter(config-isakmp)#lifetime 86400

VPN – przykład konfigurowania (Cisco IOS)

- ⌘ Przykład prostej konfiguracji tunelu, Cisco, definiowanie kluczy i ISAKMP:
- ⌘ Klucz szyfrowania VPN wybieramy jednocześnie z określeniem przeciwnego końca tunelu. (tzw. VPN peer):
- ⌘ *Ruter(config)#crypto isakmp key 6 mojklucz address 200.200.200.2*
gdzie 6 oznacza wymuszenie szyfrowania klucza (0 – klucz nieszyfrowany)
- ⌘ Faza 2: uruchomienie IPSec:
- ⌘ Parametry szyfrowania w tunelu IPSec to pula algorytmów szyfrowania IPSec z której nastąpi uzgodnienie:
- ⌘ *Ruter(config)#crypto ipsec transform-set mojipsec esp-aes esp-sha*
Ruter(cfg-crypto-trans)#exit

IPSec



- ⌘ Technika tunelowania w obrębie sieci IP z szyfrowaniem danych przesyłanych.
- ⌘ Protokół półotwarty - definiuje ogólne reguły obsługi tunelowania, ale nie wymusza stosowania konkretnych technicznych (to definiują następne wybrane w danej konfiguracji protokoły wewnętrzne)
- ⌘ Podstawa technologiczna VPN - dwa protokoły wewnętrzne:
 - ☒ AH (*Authentication Header*) - zapewnia uwierzytelnianie stacji
 - ☒ ESP (*Encapsulating Security Payload*) - zapewnia szyfrowanie

IPSec - IKE (I)



- ⌘ Protokół IKE (Internet Key Exchange) jest rozszerzeniem ISAKMP (Internet Security Association and Key Management Protocol), zawierającym dodatkowe funkcjonalności.
- ⌘ IKE dostarcza:
 - ☒ Specyfikację metod kryptograficznych
 - ☒ Opisy metod uwierzytelnienia i negocjacji kluczy
 - ☒ Specyfikację formatów pakietów
- ⌘ Dopuszcza stosowanie różnych systemów (szyfrowanie, uwierzytelnienie) - za każdym użyciem dodawanych do protokołu z postaci tzw. DOI (*Domain of Interpretation*)
- ⌘ Umożliwia automatyczną negocjację parametrów połączenia - ustalanych na podstawie puli możliwości technicznych obydwu stron

IPSec - IKE (II)

⌘ Funkcje IKE:

- ☒ Uwierzytelnienie obu stron komunikacji wobec siebie za pomocą jednej z poniższych metod:
 - ☒ Hasło znane obu stronom (*Shared Secret*)
 - ☒ Podpisy RSA (konieczna wymiana kluczy publicznych stron)
 - ☒ Certyfikaty X.509
- ☒ Nawiązanie bezpiecznego kanału dla potrzeb IKE nazywanego ISAKMP SA (*Internet Security Association and Key Management Protocol Security Association*)
- ☒ Bezpieczne uzgodnienie kluczy kryptograficznych oraz parametrów tuneli IPSec
- ☒ Ewentualna renegotiacja treści kluczy co określony czas.

Nagłówek IPSec



- ⌘ Minimalizacja informacji jawnej - nagłówek zawiera tylko dwie pozycje:
 - ☒ SPI (*Security Parameters Index*) - 12 bitów, identyfikujących kanał pomiędzy stacjami. Liczba ta jest generowana losowo i uzgadniania przez stacje podczas tworzenia kanału.
 - ☒ Numer sekwencyjny - 32 bity, rozróżniających pakiety przesyłane w ramach danego kanału. Numer sekwencyjny zwiększany jest o 1 z każdym kolejnym pakietem.
- ⌘ Transmisja jest prowadzona na bazie kanałów IPSec, charakteryzowanych poprzez IP stacji końcowej i docelowej, oraz unikatowa wartość SPI. Kanały określane są skrótem SA (Security Association).

IPSec - kanały komunikacyjne i klucze

- ⌘ IPSec jest jednokierunkowe
- ⌘ Do komunikacji dwustronnej wymagane są dwa przeciwbieżne kanały. Każdy z nich jest indywidualnie konfigurowany i zabezpieczany.
- ⌘ W przypadku użytkowania protokołu AH wymagane są dodatkowe dwa kanały, stosowane do ochrony integralności i uwierzytelnienia
- ⌘ W przypadku stosowania AH użytkowane będą następujące klucze (zakładamy komunikację pomiędzy stacjami X i Y):
 - ⏏ Szyfrujący dla kanału relacji X do Y
 - ⏏ Uwierzytelniający dla kanału relacji X do Y
 - ⏏ Szyfrujący dla kanału relacji Y do X
 - ⏏ Uwierzytelniający dla kanału relacji Y do X
- ⌘ Wszystkie klucze są wymieniane w ustalonym interwale czasowym.

Tryby komunikacji IPSec (I)

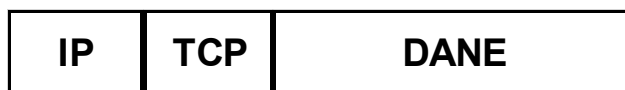


⌘ Wyróżniamy:

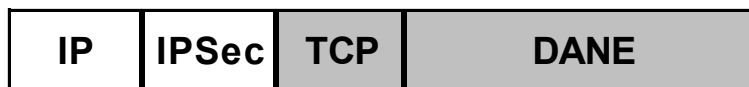
- ☒ Tryb transportowy - połączenie najczęściej stosowane pomiędzy indywidualnym hostem a zdalną siecią, lub pomiędzy dwoma indywidualnymi hostami. Szyfrowana jest tylko treść przekazywana, nie jest szyfrowana informacja o stacjach (adresy)
- ☒ Tryb tunelowy - połączenie stosowane pomiędzy bramkami łączącymi sieci „prywatne”. Szyfrowana jest także informacja o adresowaniu sieci.

Tryby komunikacji IPSec (II)

⌘ Zwykły pakiet TCP/IP:



⌘ W przypadku trybu transportowego:



⌘ W przypadku trybu tunelowego:



SSL (I)



- ⌘ SSL - Secure Socket Layer
- ⌘ Opracowany przez firmę Netscape - pierwotnie na potrzeby zabezpieczania WWW
- ⌘ Zapewnia bezpieczeństwo komunikacji w trzech aspektach:
 - ⏏ Uwierzytelnienie - zdolność do weryfikowania tożsamości zarówno serwera i klienta,
 - ⏏ Poufność - zdolność do szyfrowania przesyłanych informacji,
 - ⏏ Integralność - zapewnienie, że odbierane dane nie zostały po drodze zmienione.

SSL (II)



- ⌘ SSL funkcjonuje w warstwie transportowej, więc może wspomagać protokoły warstw wyższych
- ⌘ 3 wersje - ewoluujące pod kątem zabezpieczeń (SSL1, SSL2, SSL3 (obecna, znana też jako TLS - *Transport Layer Security*))
- ⌘ Podstawowe zasady działania:
 - ☒ Każda ze stron generuje parę kluczy (publiczny+prywatny)
 - ☒ Każda ze stron publikuje swój klucz publiczny w sieci (jest on dostępny dla wszystkich)
 - ☒ Jeżeli A chce przesłać wiadomość do B to szyfruje ją kluczem publicznym B. Z własności algorytmu wynika, że tylko używając klucza prywatnego B można ją odtworzyć (typowy schemat wymiany danych).

SSL (III)



- ⌘ Przy identyfikacji stron SSL opiera się na tzw. certyfikatach standardu X.509
- ⌘ Certyfikat to podpisany cyfrowo pakiet weryfikujący (np. dokument, stronę komunikacji itp.). Podpis (w praktyce będący szyfrem autoryzującym) jest dokonywany przez stronę trzecią, którą jest obdarzona zaufaniem (Certificate Authority)
- ⌘ Komercyjne wykorzystanie certyfikatów X.509 - generatory zamówień certyfikatów, instytucje uwierzytelniające, wzajemność uwierzytelnień (instytucje uwierzytelniające też są uwierzytelniane)

SSH (I)



- ⌘ Alternatywa dla TELNET lecz z dalszą, rozbudowaną funkcjonalnością (np. tunele)
- ⌘ Połączenie szyfrowane klient-server - w technice emulacji terminala
- ⌘ Każdy klient i serwer posiadają swoją parę kluczy -prywatny i publiczny. Klucz publiczny dostępny jest dla wszystkich użytkowników sieci
- ⌘ Para kluczy umożliwia zaszyfrowanie informacji kluczem publicznym i odszyfrowanie jej wyłącznie kluczem prywatnym, ale również zaszyfrowanie kluczem prywatnym i odszyfrowanie wyłącznie kluczem publicznym.

SSH (II)



⌘ Procedura nawiązywania sesji:

- ☒ Klient pobiera klucz publiczny serwera, sprawdza uwierzytelnienie serwera - jeśli znajdzie w swojej bazie inny klucz tego serwera (tzw. *FingerPrint*) - rozłącza się,
- ☒ Klient generuje losową liczbę 256 bitową - klucz sesji,
- ☒ Klient liczbę tę szyfruje przy pomocy swojego klucza prywatnego i następnie przy pomocy klucza publicznego serwera i tak zakodowaną przesyła do serwera,
- ☒ Serwer przy pomocy swojego klucza prywatnego i klucza publicznego klienta rozkodowuje wylosowaną liczbę, która stanowi klucz kodowy w dalszej szyfrowanej komunikacji.

X.509 - generowanie certyfikatu



- ⌘ Użyte (poglądowo) narzędzie: OpenSSL
- ⌘ Procedura tworzenia certyfikatu:
 - ☑ Przygotowanie klucza prywatnego (strony uwierzytelnianej) - klucz jest używany do generowania certyfikatu, i obsługi późniejszych transmisji szyfrowanych
 - ☑ Przygotowanie zamówienia certyfikatu (z danymi strony uwierzytelnianej)
 - ☑ Wygenerowanie certyfikatu (w CA - Certificate Authority)
Instalacja certyfikatu i klucza prywatnego (zależnie od rodzaju oprogramowania korzystającego z bezpiecznego kanału)

X.509 - generowanie certyfikatu

⌘ Pierwszy krok - wytworzenie klucza prywatnego i zamówienia

⌘ `openssl req -config openssl.cnf
-new -out server1.csr`

⌘ -----BEGIN CERTIFICATE REQUEST-----
⌘ MIIB3jCCAUCCAQAwwYYxCzAJBgNVBAYTANBMRMwEQYDVQQIEwptYWxvcG9sc2th
⌘ MQ8wDQYDVQQHEwZjcmFjb3cxDTALBgNVBAoTBG1pdHUxCzAJBgNVBAsTAm5hMRUw
⌘ EwYDVQQDEwXNaWNoYWwgVHVyZWsxHjAcBgkqhkiG9w0BCQEW21pdHVAYWdoImVh
⌘ dS5wbDCBnzANBgkqhkiG9w0BAQEFAAOBjQAwgYkCgYEA6Kc+qyeYbrfhY9+2u7JM
⌘ 8j/tX0uLyQir1tui2uV6PB3S7c3SEMCaq+SCZ/xoptau2Plyg8I8mVOD0aqcjLDz
⌘ 1tXO8X5gDcTCitKS+V8JvMwu8zw8aZsbvZ9vWDvoBO0yO9ZPzus5otlHupfgGSL/
⌘ XbA5lGlKau/zbRu4piRhMvCCAwEAAaAXMBUGCSqGSIB3DQEJBzEIEwYxMjM0NTYw
⌘ DQYJKoZIhvcNAQEFBQADgYEAQqJs/wHkZDK1jjEht3ans/UvMNPvAy9yFjPH3H
⌘ r0SaJd7fm/tzTokTsp4r09fIp3pRteqZtSNrEs5M2+hQSlAe+IV7f4ummHkHhDii
⌘ 8jrTV/IVcmzcFfXJM6ubMRu0OREcREPBehT5cGbRj/5gGPni7dfgO4jSEvhz4Pbr
⌘ jFQ=
⌘ -----END CERTIFICATE REQUEST-----

X.509 - generowanie certyfikatu



⌘ Dane identyfikacyjne do zamówienia:

- ⌘ Loading 'screen' into random state - done
- ⌘ Generating a 1024 bit RSA private key
- ⌘ writing new private key to 'privkey.pem'
- ⌘ Enter PEM pass phrase:
- ⌘ Verifying - Enter PEM pass phrase:
- ⌘ Country Name (2 letter code) [AU]:pl
- ⌘ State or Province Name (full name) [Some-State]:malopolska
- ⌘ Locality Name (eg, city) []:cracow
- ⌘ Organization Name (eg, company) [Internet Widgits Pty Ltd]:mitu
- ⌘ Organizational Unit Name (eg, section) []:na
- ⌘ Common Name (eg, YOUR name) []:Michal Turek
- ⌘ Email Address []:mitu@agh.edu.pl
- ⌘ A challenge password []:123456
- ⌘ An optional company name []:

X.509 - generowanie certyfikatu

- ⌘ Po wytworzeniu zamówienia otrzymujemy dwa pliki: .crt (zamówienie) oraz .pem (klucz prywatny). Dodatkowo wygenerowany zostanie plik „.rnd” z sekwencją losową, która razem z podanym „Pass-Phase” służyła do wygenerowania klucza. Należy go usunąć.
- ⌘ „Pass-Phase” jest dodana do pliku klucza prywatnego. Można ją usunąć (nie będzie potrzebna) komendą:

```
openssl rsa -in privkey.pem -out serwer1.key
```

- ⌘ W pliku serwer1.key znajduje się gotowy klucz prywatny dla serwera

X.509 - generowanie certyfikatu

- ⌘ Na podstawie zamówienia zostanie przygotowany podpisany kluczem prywatnym certyfikat. Treść klucza prywatnego musi być przechowywana w ścisłej tajemnicy.

-----BEGIN RSA PRIVATE KEY-----

⌘ MIICXAIBAAKBgQDopz6rJ5hut+Fj37a7skzyP+1fS4vJCKvW26La5Xo8HdLtzdIQ
⌘ wJqr5IJn/Gim1q7Y+XKDwjyZU4PRqpyMsPPW1c7xfmANxMKK0pL5Xwm8zC7zPDxp
⌘ mxu9n29YO+gE7TI71k/O6zmi2Ue6l+AZIv9dsDmUaUpq7/NtG7imJGEy9wIDAQAB
⌘ AoGAI3BudGXoqlzpQWxvinHj1Lc2MmDwnyPqASWdzhhMoWGtornKePDN3sdFMg/Z
⌘ 5SgopDaDQhzRXE3Dc6m4hRnjWpkDxDaoQ50XjVIVEJdm11VvqSeGP6I71tTm20W2
⌘ 0GHD21Ei0cIXg62NtjSQjrT3Y5TsLD6JxSdw9DiIOUQIbuECQQD4ySGTa+BmPfGb
⌘ 8xEVqHP9eCCX5zbPYvYvJy7Sh11Fs0/cbju2KBptdv/dYm9uouuohcRMDsXw3aLL
⌘ DfGJWJ6nAkeA72Zavch/zeJDkrn+yi7mzAwfuMk7TA5Me/UstH1JRUZTAyKPAQfY
⌘ c/xUZ0iS3kky3KcxKC6z0CEBVXN1vrYjMQJAFHIjKkRxWrW4h4KAYvFHJwuaUiGU
⌘ ZZLSfKmRtYeJCqtj4s/EZIIIDof/Po1gVpFgLWfw0V7PXHKZ6XIm20fzQfQJAbXhQ
⌘ ZhgY2PxuCdOjODDaONQSBv0XUF/5eXVTS/U+W/8cVoKnAcGkqOTpc5EsF51BOMT7
⌘ JXS1pKZzBbAPBnYjAQJBAOBL96jggEITS6mLescMVF+TSMkYZzEIyfkvbG5C1wOp
⌘ Ah0h//H85o0tgpDpFqtXJqndzZjGBGE91SM05nCB1m0=
⌘ -----END RSA PRIVATE KEY-----

X.509 - generowanie certyfikatu

- ⌘ Możliwe jest własnoręczne (tymczasowe) utworzenie certyfikatu na podstawie zamówienia i klucza -jednak przez nikogo nie uwiarygodnionego:

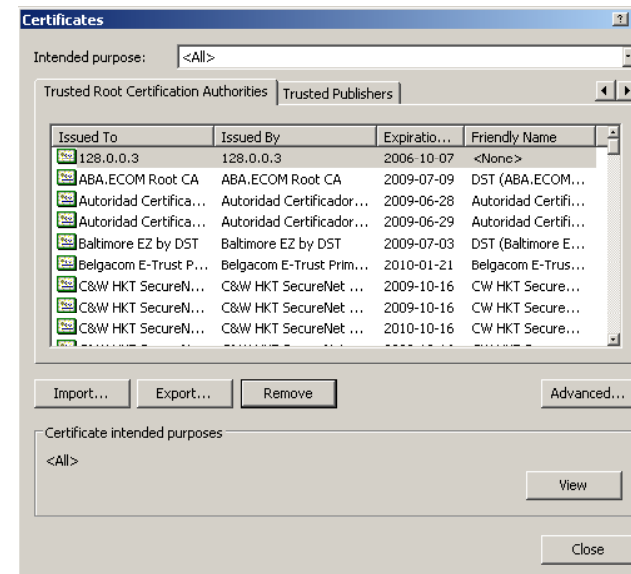
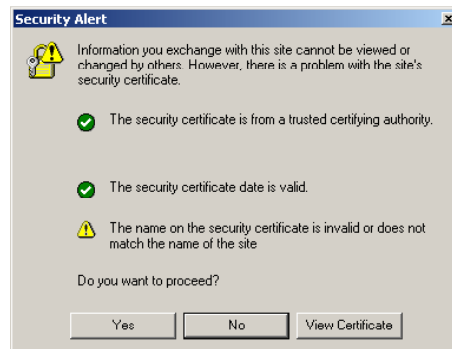
```
openssl x509 -in serwer1.csr -out serwer1.cert  
-req -signkey serwer1.key -days 365
```

- ⌘ Oraz jego odpowiednika dla IE (format binarny DER):

```
openssl x509 -in serwer1.cert  
-out serwer1.der.crt -outform DER
```

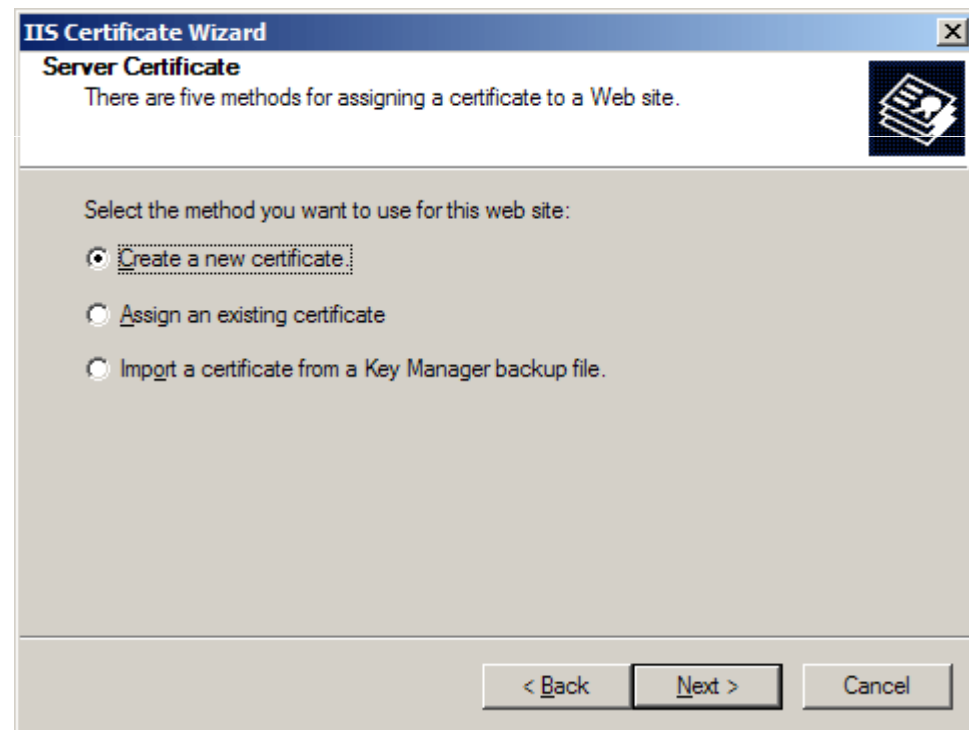
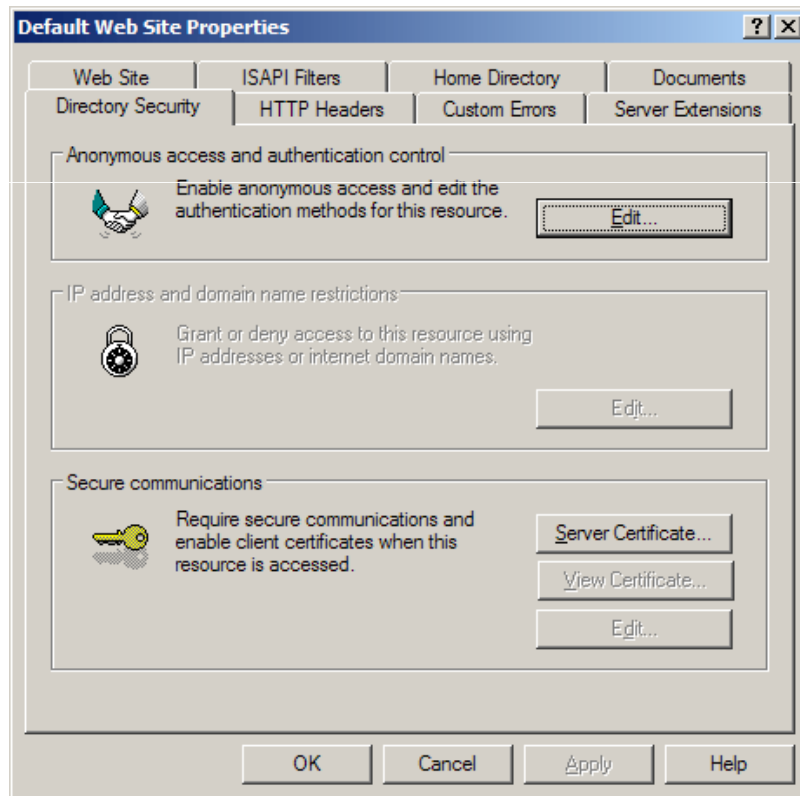
X.509 - generowanie i import certyfikatu

- ⌘ Certyfikat będzie pobierany z serwera WWW przez przeglądarkę. Razem z kluczem prywatnym należy go dostarczyć do serwera
- ⌘ Po weryfikacji certyfikatu następuje nawiązanie połączenia szyfrowanego HTTPS
- ⌘ HTTPS funkcjonuje nad portem 443 TCP.
- ⌘ Ewentualne problemy w imporcie - zgłaszane użytkownikowi.
- ⌘ Gdy certyfikat jest uwiarygodniony:



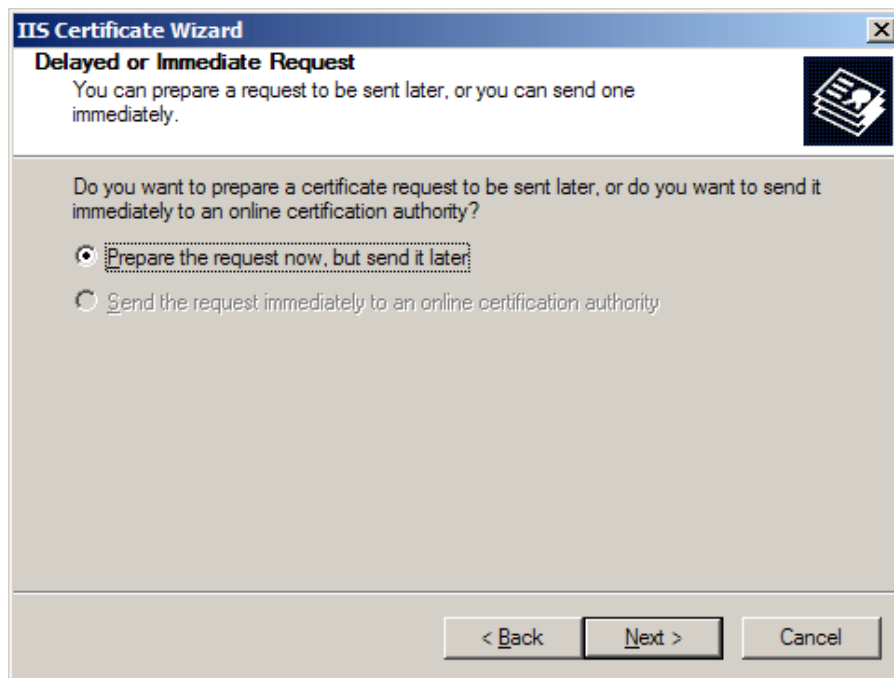
IIS - przygotowanie i instalacja certyfikatu (I)

✂ Gotowy interfejs do generowania zamówienia (IIS ver. 5)



IIS - przygotowanie i instalacja certyfikatu (II)

- ⌘ Zamówienie zapisywane jest w pliku, dane do zamówienia pobierane przez konsolę MMC IIS.
- ⌘ Brak uwiarygodnienia certyfikatu umożliwia połączenia szyfrowane lecz nie gwarantuje prawdziwej tożsamości serwera



IIS Certificate Wizard

Delayed or Immediate Request

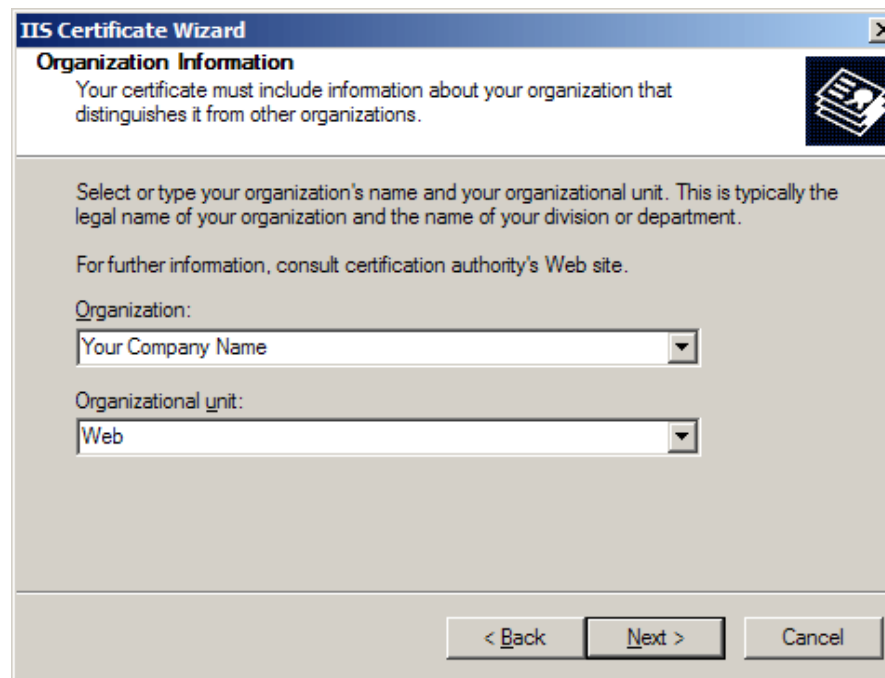
You can prepare a request to be sent later, or you can send one immediately.

Do you want to prepare a certificate request to be sent later, or do you want to send it immediately to an online certification authority?

☒ Prepare the request now, but send it later

☐ Send the request immediately to an online certification authority

< Back Next > Cancel



IIS Certificate Wizard

Organization Information

Your certificate must include information about your organization that distinguishes it from other organizations.

Select or type your organization's name and your organizational unit. This is typically the legal name of your organization and the name of your division or department.

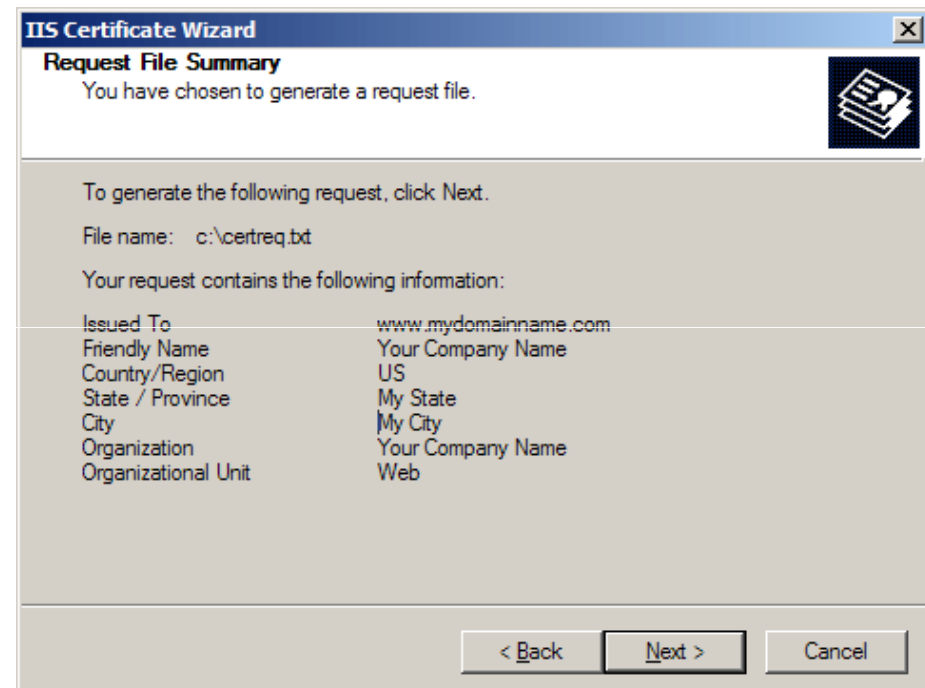
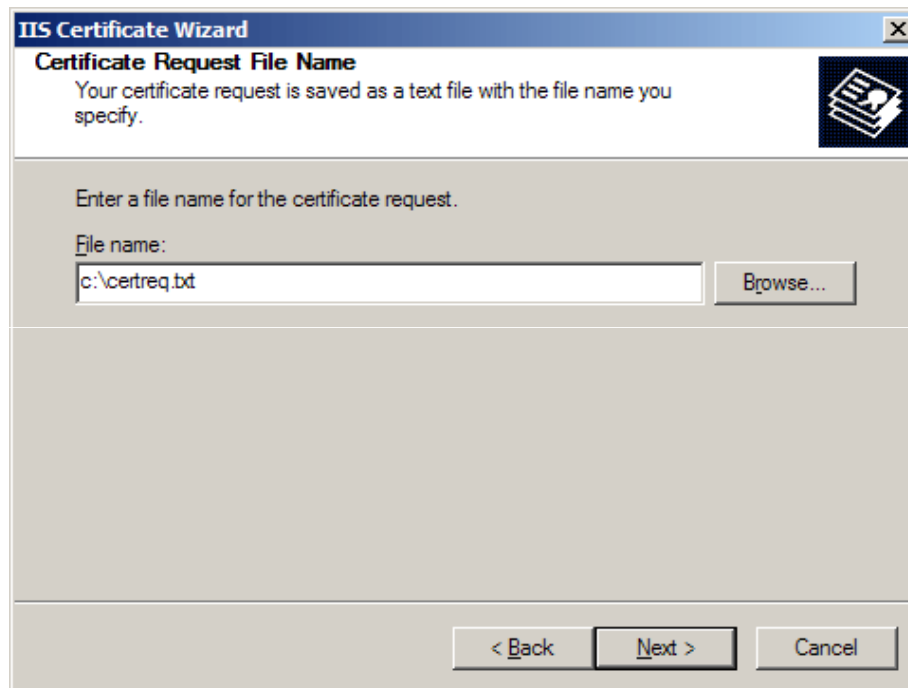
For further information, consult certification authority's Web site.

Organization:
Your Company Name

Organizational unit:
Web

< Back Next > Cancel

IIS - przygotowanie i instalacja certyfikatu (III)



- ⌘ Zapisanie klucza prywatnego (backup): Certificates -> Requests -> All tasks -> Export

Apache - przygotowanie i instalacja certyfikatu (I)

⌘ Konieczne jest dodanie modułów SSL, jeśli w edycji serwera takich nie ma. Należy wtedy:

- ⏏ Pobrać moduł : mod_ssl
- ⏏ Umieścić biblioteki ssleay32.dll i libeay32.dll w systemie (dla Windows)
- ⏏ Umieścić w pliku httpd.conf linię ładującą moduł:

```
LoadModule ssl_module  
modules/ApacheModuleSSL.dll
```

Apache - przygotowanie i instalacja certyfikatu (II)

⌘ Dodanie modułów SSL (cd):

☑ W pliku httpd.conf należy też skonfigurować parametry SSL i virtual host podlegający szyfrowaniu:

```
SSLRandomSeed startup builtin
SSLSessionCache none
SSLLog logs/SSL.log
SSLLogLevel info
```

```
<VirtualHost moj_host:443>
SSLEngine On
SSLCertificateFile conf/ssl/serwer1.cert
SSLCertificateKeyFile conf/ssl/serwer1.key
</VirtualHost>
```

Apache - przygotowanie i instalacja certyfikatu (III)



- ⌘ W przypadku posiadania serwera z już zainstalowanym SSL - należy uruchomić go z opcją *-D SSL*
- ⌘ Instalacja certyfikatu - ręczna, poprzez dodanie plików do katalogu
- ⌘ W katalogu conf/ssl (lub innym - zgodnie z konfiguracją) należy umieścić plik klucza prywatnego i certyfikat

Ekranowanie sieci (I)

- ⌘ Cel stosowania – izolowanie segmentu sieci utrudniające ingerencję w jej zasoby, komunikowanie VPN z odległymi segmentami sieci, kontrola własnych użytkowników, prowadzenie statystyk.
- ⌘ Środki: Security Appliance (Firewall wielofunkcyjny), rodzaje:
 - ☒ Rozwiązania programowe - oprogramowanie kontrolne dla systemów operacyjnych,
 - ☒ Rozwiązania budowane na bazie specjalnie zaprojektowanego sprzętu (routera ekranującego)
- ⌘ W przypadku konieczności świadczenia usług z wewnątrz segmentu sieci chronionej:
 - ☒ Przeniesienie usług na firewall (szczególnie w przypadku rozwiązań programowych)
 - ☒ Wyznaczenie DMZ (DeMilitarized Zone) - maszyny wyłączonej z ochrony (czasem ograniczonej w komunikacji np. zakresem dozwolonych portów TCP).

Ekranowanie sieci (II)

⌘ Trzy typy funkcjonowania:

- ☑ Filtracja pakietów (*Packet-filtering Firewall*) - analiza pakietów jedynie pod kątem typu. Może odbywać się w dolnych warstwach OSI.
- ☑ Filtracja z inspekcją stanów połączeń i statusów stron komunikacji (*Stateful-inspection Firewall*) - analiza treści, w tym adresów źródła, przeznaczenia, wyszukiwanie pod kątem treści (np. wykrywanie podłańcuchów), wykorzystanie informacji o poprzednich pakietach (stan sesji, analiza przebiegu komunikacji w protokole warstwy wyższej) itp.
- ☑ Pośredniczenie w sesji (Application Proxy Firewall) - autoryzacja użytkowników, upoważnionych do nawiązywania określonych połączeń w warstwach wyższych.

Security Appliance - typowe produkty

- ⌘ Rynek rozwiązań sprzętowych, przykłady:: Cisco Secure Private Internet eXchange PIX), Cisco ASA; Symantec Firewall, IBM Proventia; Check Point – Nokia; FireBox; Juniper SSG, SRX, Netscreen
- ⌘ Często dostosowanie do wyszukanych zapotrzebowań rynkowych i zagrożeń z nimi związanych
- ⌘ Wysoka wydajność wymagająca także dużej ilości pamięci (utrzymywanie historii sesji, wzorców sytuacji zagrażających itp.)
- ⌘ W większości przypadków sprzęt budowany jest w oparciu o platformę PC (umieszczoną fizycznie w obudowie RACK i rozbudowaną o dodatkowe porty Ethernet) natomiast oprogramowanie tworzone jest jako nakładka na system operacyjny PC (najczęściej Linux)
- ⌘ Konfigurowanie: przez interfejs WWW lub dedykowaną aplikację kliencką komunikującą się z urządzeniem



IDS – Intrusion Detection Systems



- ⌘ IDS (*Intrusion Detection System*) – główne zastosowanie: do wykrywania ataków lecz bez dostarczania mechanizmów ochrony przed ich skutkami
- ⌘ Jedna z technik nadzorowania bezpieczeństwa sieci
- ⌘ Podstawowa metoda: analiza ruchu sieciowego z dopasowywaniem do zdefiniowanych wzorców (sygnatur) lub przez poszukiwanie i raportowanie ogólnych nieprawidłowości komunikacyjnych.
- ⌘ Niektóre IDS oparte są o zdolne do nauki algorytmy heurystyczne, które automatycznie dostosowują się do danej sieci
- ⌘ Zarządzanie: interfejsy WWW. Spora część IDS jest budowana w oparciu o platformy PC – umieszczone fizycznie w karcie przełącznika CORE (*line-card*) lub w module rozszerzenia do rutera.

IDS – rozwiązania sprzętowe



- ⌘ Funkcjonują na poziomie całej sieci lub jej wybranego segmentu, monitorując znajdujący się tam ruch
- ⌘ Są rozszerzeniami przełączników CORE lub ruterów pośredniczących w ruchu wyprowadzanym na zewnątrz
- ⌘ Architektura przełącznika CORE pozwala na prowadzenie monitorowania bez dodatkowego obciążania magistrali, co jest głównym argumentem za montowaniem IDS w tych urządzeniach
- ⌘ IDS działają pod kontrolą osobnego systemu operacyjnego - ze sporą ilością pamięci przeznaczonej do przechowywania sygnatur operacji zabronionych
- ⌘ Producent aktualizuje IDS dostarczając nowe bazy sygnatur
- ⌘ Możliwość monitorowania ruchu w sieci z uwzględnieniem stanu połączenia

Przykład oprogramowania

IDS: System Snort



- ⌘ System Snort to przykładowy IDS oraz sniffer. Wymaga instalowania na serwerze pełniącego rolę bramki na zewnątrz lub otrzymującego kopie ruchu w sieci
- ⌘ Popularne rozwiązanie Open Source
- ⌘ Umożliwia w czasie rzeczywistym dokonywanie analizy ruchu i rejestrowanie pakietów przechodzących przez sieci oparte na protokołach IP/TCP/UDP/ICMP
- ⌘ Diagnostyka przede wszystkim ataki przepełnienia bufora, skanowania portów typu stealth, na usługi WWW, SMB, czy próby wykrywania systemu operacyjnego, próby użytkownika oprogramowania typu exploit do znanych luk w usługach sieciowych